Actividad 2: Lógica de Programación y Programación Orientada a Objetos

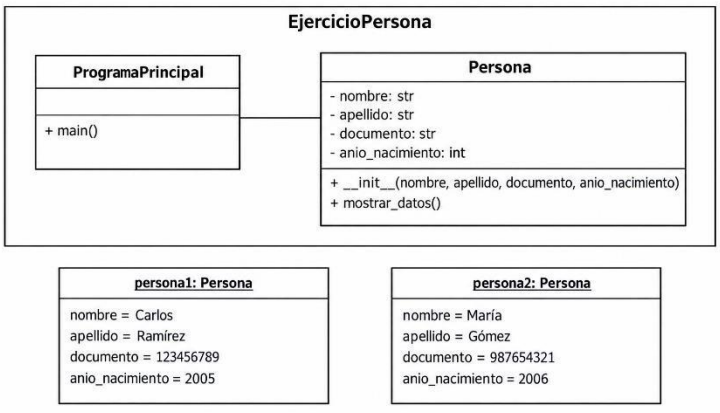
Universidad Nacional de Colombia Sede Medellin
Docente: Walter Hugo Arboleda Mazo

Brayan Caro Viana
@bcarov@unal.edu.co

Ejercicio 2.1 – Clase persona

```
 class Persona:     def __init__(self, nombre, apellido, documento,
anio_nacimiento):
        self.nombre = nombre
self.apellido = apellido
self.documento = documento
self.anio_nacimiento = anio_nacimiento
     def
mostrar_datos(self):
        print("Nombre:", self.nombre)
print("Apellido:", self.apellido)
print("Documento de identidad:", self.documento)
print("Año de nacimiento:", self.anio_nacimiento)
print("----------------------------")
 def
main():
    # Crear dos personas     persona1 = Persona("Carlos",
"Ramírez", "123456789", 2005)     persona2 = Persona("María",
"Gómez", "987654321", 2006)
    # Mostrar los datos de las personas
print("DATOS DE LA PERSONA 1")
persona1.mostrar_datos()
     print("DATOS DE LA PERSONA
2")     persona2.mostrar_datos()
 if __name__ ==
"__main__":
    main()
```

Diagrama de Clase:



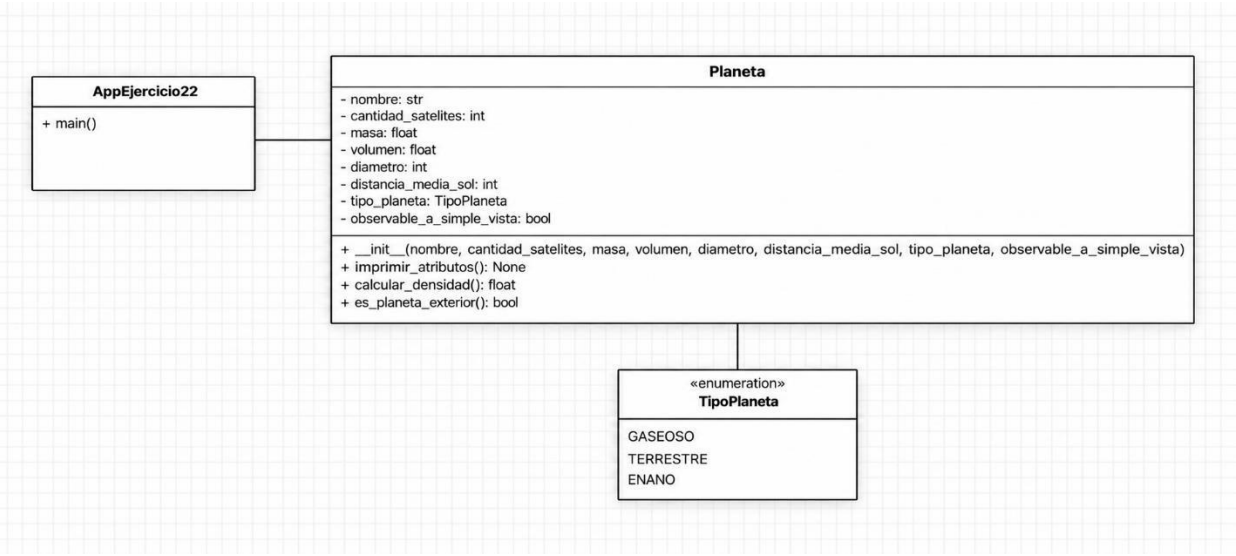
Ejercicio 2.2 – Planetas

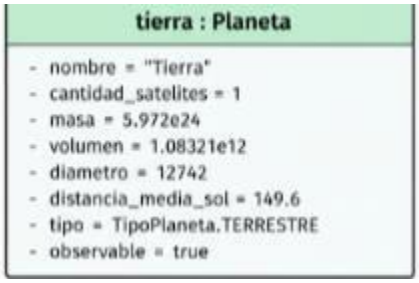
```
from enum import Enum
 class
TipoPlaneta(Enum):
GASEOSO = "GASEOSO"
    TERRESTRE = "TERRESTRE"
    ENANO = "ENANO"
 class Planeta:     # Constructor     def __init__(
self,         nombre=None,
cantidad_satelites=0,         masa=0.0,
volumen=0.0,         diametro=0,
distancia_media_sol=0,         tipo=None,
observable=False     ):         self.nombre = nombre
self.cantidad_satelites = cantidad_satelites
self.masa = masa         self.volumen = volumen
self.diametro = diametro
self.distancia_media_sol = distancia_media_sol
self.tipo = tipo         self.observable = observable
     def
imprimir_datos(self):
        print("Nombre:", self.nombre)
        print("Cantidad de satelites:", self.cantidad_satelites)
print("Masa (kg):", self.masa)
        print("Volumen (km cubicos):", self.volumen)
print("Diametro (km):", self.diametro)
print("Distancia media al Sol (millones de km):",
self.distancia_media_sol)         print("Tipo de
planeta:",
              self.tipo.value if self.tipo else "No especificado")
print("Observable a simple vista:",
              "Si" if self.observable else "No")
             def
calcular_densidad(self):
if self.volumen == 0:
            return 0         return
self.masa / self.volumen
```

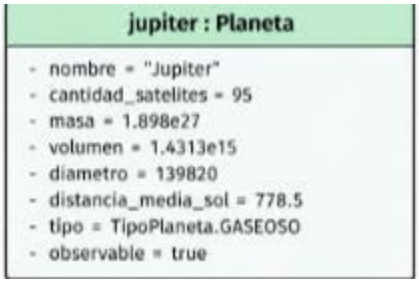
```
    def es_planeta_exterior(self):         UA_KM = 149597870         distancia_km =
self.distancia_media_sol * 1_000_000          return distancia_km > 3.4 * UA_KM
     def main():
    # Crear dos planetas     planeta1 = Planeta(
        "Tierra",         1,
        5.972e24,
        1.08321e12,
        12742,
        149.6,
        TipoPlaneta.TERRESTRE,
        True
    )      planeta2 = Planeta(
        "Jupiter",
        95,
        1.898e27,
        1.4313e15,
        139820,
        778.5,
        TipoPlaneta.GASEOSO,
        True
    )      planetas = [planeta1, planeta2]
     for planeta in planetas:
        print("\n===== DATOS DEL PLANETA =====")         planeta.imprimir_datos()
         print("Densidad (kg/km cubicos):",
planeta.calcular_densidad())
         print("Es un planeta exterior:",
              "Si" if planeta.es_planeta_exterior() else "No")


if __name__ == "__main__":
    main()
```

Diagrama de Clase:







Ejercicio 2.3 – Clase automovil

```
from enum import Enum

# --- Definición de Enumeraciones ---
class TipoCombustible(Enum):
GASOLINA = "Gasolina"
    BIOETANOL = "Bioetanol"
DIESEL = "Diésel"
    BIODIESEL = "Biodiésel"
    GAS_NATURAL = "Gas natural"


class TipoAutomovil(Enum):
    CARRO_CIUDAD = "Carro de ciudad"
    SUBCOMPACTO = "Subcompacto"
    COMPACTO = "Compacto"
    FAMILIAR = "Familiar"
    EJECUTIVO = "Ejecutivo"
    SUV = "SUV"


class Color(Enum):
BLANCO = "Blanco"
    NEGRO = "Negro"
    ROJO = "Rojo"
    NARANJA = "Naranja"
    AMARILLO = "Amarillo"
    VERDE = "Verde"
    AZUL = "Azul"
    VIOLETA = "Violeta"


# --- Definición de la Clase Automóvil --
- class Automovil:     def __init__(
self,         marca: str,         modelo:
int,         motor: float,
tipo_combustible: TipoCombustible,
tipo_automovil: TipoAutomovil,
numero_puertas: int,
cantidad_asientos: int,
velocidad_maxima: float,         color:
Color,         velocidad_actual: float =
0.0,
```

```
    ):
        self._marca = marca         self._modelo =
modelo         self._motor = motor
self._tipo_combustible = tipo_combustible
self._tipo_automovil = tipo_automovil
self._numero_puertas = numero_puertas
self._cantidad_asientos = cantidad_asientos
self._velocidad_maxima = velocidad_maxima
self._color = color         self._velocidad_actual
= velocidad_actual

    # --- Getters y Setters ---
    @property
def marca(self):
        return self._marca

    @marca.setter     def
marca(self, valor):
self._marca = valor

    @property
def modelo(self):
        return self._modelo

    @modelo.setter     def
modelo(self, valor):
self._modelo = valor

    @property
def motor(self):
        return self._motor

    @motor.setter     def
motor(self, valor):
self._motor = valor

    @property     def
tipo_combustible(self):
        return self._tipo_combustible
    @tipo_combustible.setter     def
tipo_combustible(self, valor):
self._tipo_combustible = valor
    @property     def
tipo_automovil(self):
        return self._tipo_automovil
```

```
    @tipo_automovil.setter     def
tipo_automovil(self, valor):
self._tipo_automovil = valor
    @property     def
numero_puertas(self):
        return self._numero_puertas
    @numero_puertas.setter     def
numero_puertas(self, valor):
self._numero_puertas = valor
    @property     def
cantidad_asientos(self):
        return self._cantidad_asientos
    @cantidad_asientos.setter     def
cantidad_asientos(self, valor):
self._cantidad_asientos = valor
    @property     def
velocidad_maxima(self):
        return self._velocidad_maxima
    @velocidad_maxima.setter     def
velocidad_maxima(self, valor):
self._velocidad_maxima = valor
    @property
def color(self):
        return self._color

    @color.setter     def
color(self, valor):
self._color = valor

    @property     def
velocidad_actual(self):
        return self._velocidad_actual
    @velocidad_actual.setter     def
velocidad_actual(self, valor):
self._velocidad_actual = valor
    # --- Métodos de Control de Velocidad ---
def acelerar(self, incremento: float):
        nueva_velocidad = self._velocidad_actual + incremento
```

```
        if nueva_velocidad > self._velocidad_maxima:
            print(                f"Error: No se puede acelerar a {nueva_velocidad} km/h.
Excede la velocidad máxima permitida ({self._velocidad_maxima} km/h)."
            )
else:
            self._velocidad_actual = nueva_velocidad
    def desacelerar(self, decremento:
float):
        nueva_velocidad = self._velocidad_actual - decremento
if nueva_velocidad < 0:
            print(                f"Error: No se puede desacelerar en
{decremento} km/h. La velocidad resultante no puede ser negativa."
            )
else:
            self._velocidad_actual = nueva_velocidad
    def
frenar(self):
        self._velocidad_actual = 0.0
    # --- Método para calcular tiempo estimado de llegada ---
def calcular_tiempo_llegada(self, distancia: float) -> float:
if self._velocidad_actual == 0:
            print(
                "Advertencia: El vehículo está detenido. No se puede calcular el tiempo
de llegada."             )             return float("inf")         return distancia /
self._velocidad_actual

    # --- Método para mostrar atributos ---
def mostrar_atributos(self):
        print("\n--- Datos del Automóvil ---")
print(f"Marca: {self._marca}")         print(f"Modelo
(Año): {self._modelo}")         print(f"Motor:
{self._motor} L")
        print(f"Tipo de Combustible: {self._tipo_combustible.value}")
print(f"Tipo de Automóvil: {self._tipo_automovil.value}")
print(f"Número de Puertas: {self._numero_puertas}")         print(f"Cantidad
de Asientos: {self._cantidad_asientos}")         print(f"Velocidad Máxima:
{self._velocidad_maxima} km/h")         print(f"Color: {self._color.value}")
        print(f"Velocidad Actual: {self._velocidad_actual} km/h")
print("---------------------------\n")

# --- Método Main ---
```

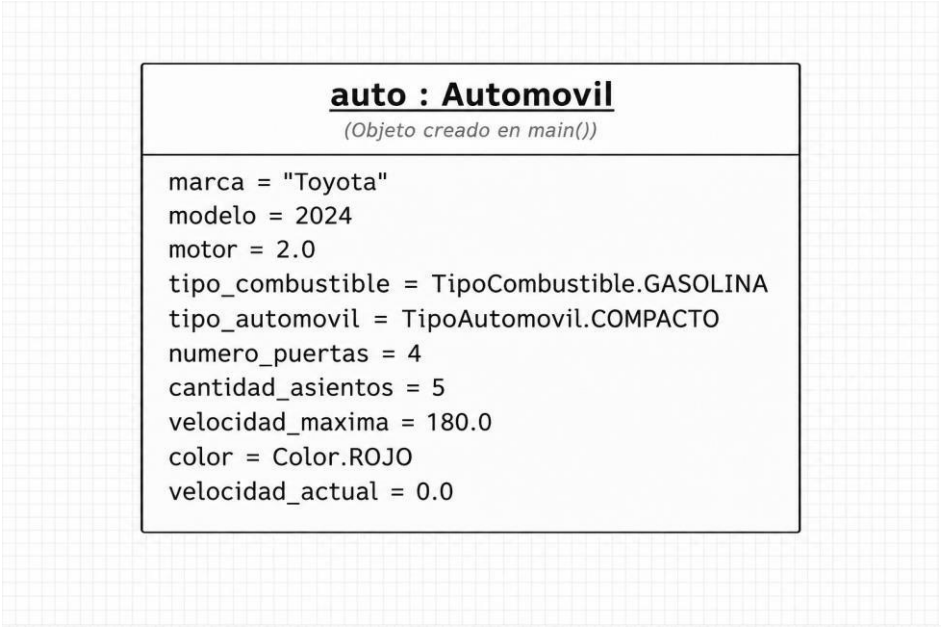
```
def main():
    # Creación del automóvil     auto =
Automovil(         marca="Toyota",
modelo=2024,          motor=2.0,
tipo_combustible=TipoCombustible.GASOLINA,
tipo_automovil=TipoAutomovil.COMPACTO,
numero_puertas=4,         cantidad_asientos=5,
velocidad_maxima=180.0,         color=Color.ROJO,
velocidad_actual=0.0,
    )

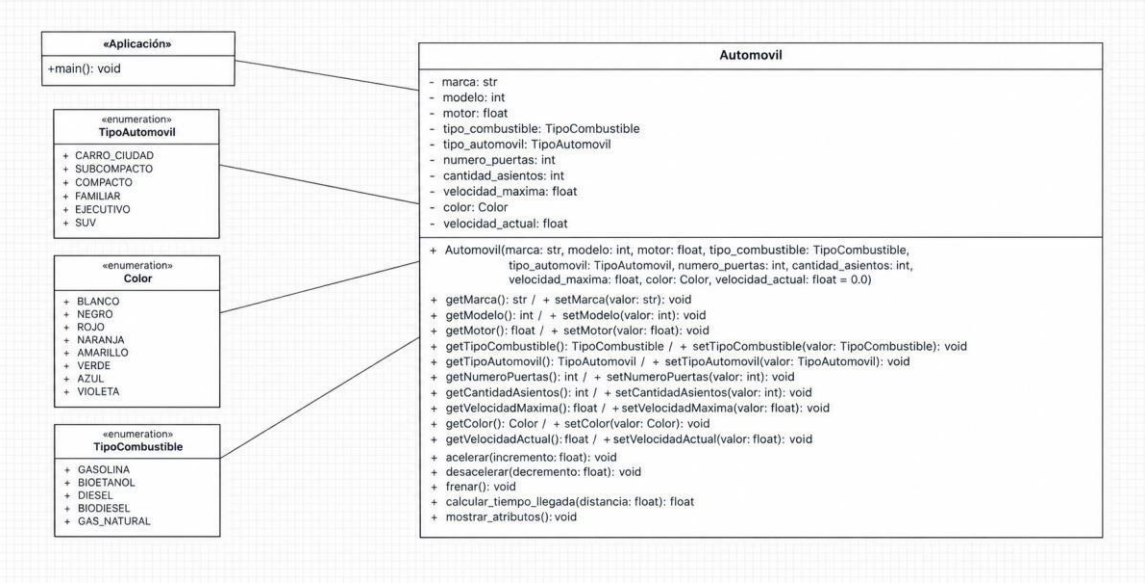
    # Mostrar datos iniciales
auto.mostrar_atributos()

    # Colocar su velocidad actual en 100 km/h
print("Estableciendo velocidad inicial en 100 km/h...")
auto.velocidad_actual = 100.0     print(f"Velocidad actual:
{auto.velocidad_actual} km/h\n")
    # Aumentar velocidad en 20 km/h     print("Acelerando 20
km/h...")     auto.acelerar(20)     print(f"Velocidad actual:
{auto.velocidad_actual} km/h\n")
    # Decrementar velocidad en 50 km/h
print("Desacelerando 50 km/h...")     auto.desacelerar(50)
    print(f"Velocidad actual: {auto.velocidad_actual} km/h\n")
    # Frenar
    print("Frenando...")     auto.frenar()
print(f"Velocidad actual: {auto.velocidad_actual} km/h\n")


if __name__ == "__main__":
    main()
```

Diagrama de Clase:





Ejercicio 2.4 - CLASE FIGURAS

```
import math

# --- Clase Círculo --- class
Circulo:     def __init__(self,
radio: float):
        self.radio = radio
     def calcular_area(self) -> float:
return math.pi * (self.radio**2)
     def calcular_perimetro(self) ->
float:         return 2 * math.pi *
self.radio

# --- Clase Rectángulo --- class Rectangulo:
def __init__(self, base: float, altura: float):
        self.base = base
self.altura = altura
     def calcular_area(self) ->
float:         return self.base *
self.altura
     def calcular_perimetro(self) -> float:
return 2 * (self.base + self.altura)

# --- Clase Cuadrado --- class
Cuadrado:
    def __init__(self, lado: float):
        self.lado = lado
     def calcular_area(self) ->
float:
        return self.lado**2
     def calcular_perimetro(self) ->
float:
        return 4 * self.lado

# --- Clase Triángulo Rectángulo --- class
TrianguloRectangulo:     def __init__(self, base:
float, altura: float):
        self.base = base
self.altura = altura
     def calcular_area(self) ->
float:
        return (self.base * self.altura) / 2
```

```
    def calcular_hipotenusa(self) ->
float:
        return math.sqrt(self.base**2 + self.altura**2)
    def calcular_perimetro(self) ->
float:
        return self.base + self.altura + self.calcular_hipotenusa()
    def determinar_tipo_triangulo(self) ->
str:
        h = self.calcular_hipotenusa()
        # Se verifica la igualdad entre los tres lados (base, altura e hipotenusa)
if self.base == self.altura == h:
            return "Equilátero"
elif (              self.base ==
self.altura             or
self.base == h             or
self.altura == h
        ):             return
"Isósceles"         else:
            return "Escaleno"

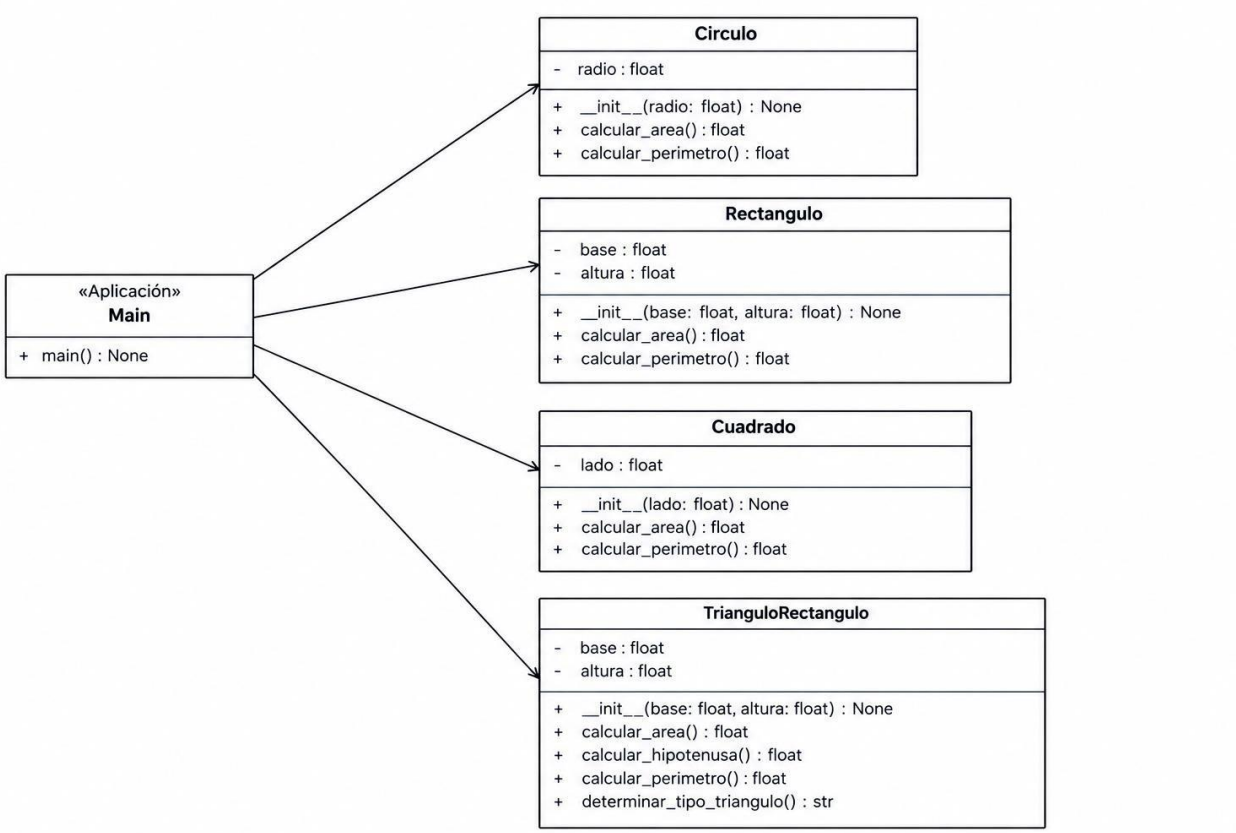
# --- Clase/Método de prueba (Main) --- def
main() -> None:
    # 1. Crear las instancias de cada figura     circulo
= Circulo(radio=5.0)     rectangulo =
Rectangulo(base=4.0, altura=6.0)     cuadrado =
Cuadrado(lado=4.0)     triangulo =
TrianguloRectangulo(base=3.0, altura=4.0)
    # 2. Probar y mostrar resultados de Círculo
print("=== CÍRCULO ===")     print(f"Radio:
{circulo.radio} cm")     print(f"Área:
{circulo.calcular_area():.2f} cm²")
    print(f"Perímetro: {circulo.calcular_perimetro():.2f} cm\n")
    # 3. Probar y mostrar resultados de Rectángulo
print("=== RECTÁNGULO ===")
    print(f"Base: {rectangulo.base} cm | Altura: {rectangulo.altura}
cm")     print(f"Área: {rectangulo.calcular_area():.2f} cm²")
print(f"Perímetro: {rectangulo.calcular_perimetro():.2f} cm\n")
    # 4. Probar y mostrar resultados de Cuadrado
print("=== CUADRADO ===")     print(f"Lado:
{cuadrado.lado} cm")     print(f"Área:
{cuadrado.calcular_area():.2f} cm²")
    print(f"Perímetro: {cuadrado.calcular_perimetro():.2f} cm\n")
    # 5. Probar y mostrar resultados de Triángulo Rectángulo
print("=== TRIÁNGULO RECTÁNGULO ===")
```

```
    print(f"Base: {triangulo.base} cm | Altura: {triangulo.altura} cm")
print(f"Área: {triangulo.calcular_area():.2f} cm²")     print(f"Hipotenusa:
{triangulo.calcular_hipotenusa():.2f} cm")     print(f"Perímetro:
```

```
{triangulo.calcular_perimetro():.2f} cm")     print(f"Tipo de triángulo:
{triangulo.determinar_tipo_triangulo()}\n")


if __name__ == "__main__":
    main()
```

Diagrama de Clase:



Ejercicio 2.5 - Clase banco

```
from enum import Enum

# --- Definición del tipo de cuenta --
- class TipoCuenta(Enum):     AHORROS
= "Ahorros"
    CORRIENTE = "Corriente"


# --- Clase CuentaBancaria --- class
CuentaBancaria:
    def __init__(         self,
nombres_titular: str,         apellidos_titular:
str,         numero_cuenta: str,
tipo_cuenta: TipoCuenta,     ):
self.nombres_titular = nombres_titular
self.apellidos_titular = apellidos_titular
self.numero_cuenta = numero_cuenta
self.tipo_cuenta = tipo_cuenta         # El saldo
inicial siempre es cero         self.saldo = 0.0

    # --- Método para imprimir valores de la cuenta ---
def imprimir_datos(self) -> None:
        print("\n--- Información de la Cuenta Bancaria ---")
print(f"Titular: {self.nombres_titular} {self.apellidos_titular}")
print(f"Número de cuenta: {self.numero_cuenta}")         print(f"Tipo de
cuenta: {self.tipo_cuenta.value}")         print(f"Saldo actual:
${self.saldo:,.2f}")
        print("-----------------------------------------\n")
    # --- Método para consultar saldo ---
def consultar_saldo(self) -> float:
        return self.saldo

    # --- Método para consignar un valor ---
def consignar(self, valor: float) -> bool:
if valor > 0:
            self.saldo += valor
print(
                f"Consignación exitosa de ${valor:,.2f}. Nuevo saldo: ${self.saldo:,.2f}"
            )
return True
```

```
        else:
            print("Error: El valor a consignar debe ser mayor a cero.")
return False

    # --- Método para retirar un valor ---     def retirar(self,
valor: float) -> bool:         if valor <= 0:
print("Error: El valor a retirar debe ser mayor a cero.")
return False         elif valor > self.saldo:
            print(                f"Error de retiro: El valor solicitado (${valor:,.2f})
supera el saldo actual
(${self.saldo:,.2f})."
            )
return False
else:
            self.saldo -= valor             print(                 f"Retiro
exitoso de ${valor:,.2f}. Nuevo saldo: ${self.saldo:,.2f}"
            )
            return True

# --- Clase de prueba / Método Main --- def
main() -> None:
    # 1. Crear una nueva cuenta bancaria (saldo inicial
0)     cuenta = CuentaBancaria(
nombres_titular="Carlos",
apellidos_titular="Gómez Pérez",
numero_cuenta="1234567890",
tipo_cuenta=TipoCuenta.AHORROS,
    )

    # 2. Imprimir los datos iniciales
cuenta.imprimir_datos()

    # 3. Consultar el saldo inicial     print(f"Saldo consultado:
${cuenta.consultar_saldo():,.2f}\n")
    # 4. Realizar una consignación
print("Intentando consignar $500,000...")
cuenta.consignar(500000)

    # 5. Intentar realizar un retiro válido
print("\nIntentando retirar $200,000...")
cuenta.retirar(200000)

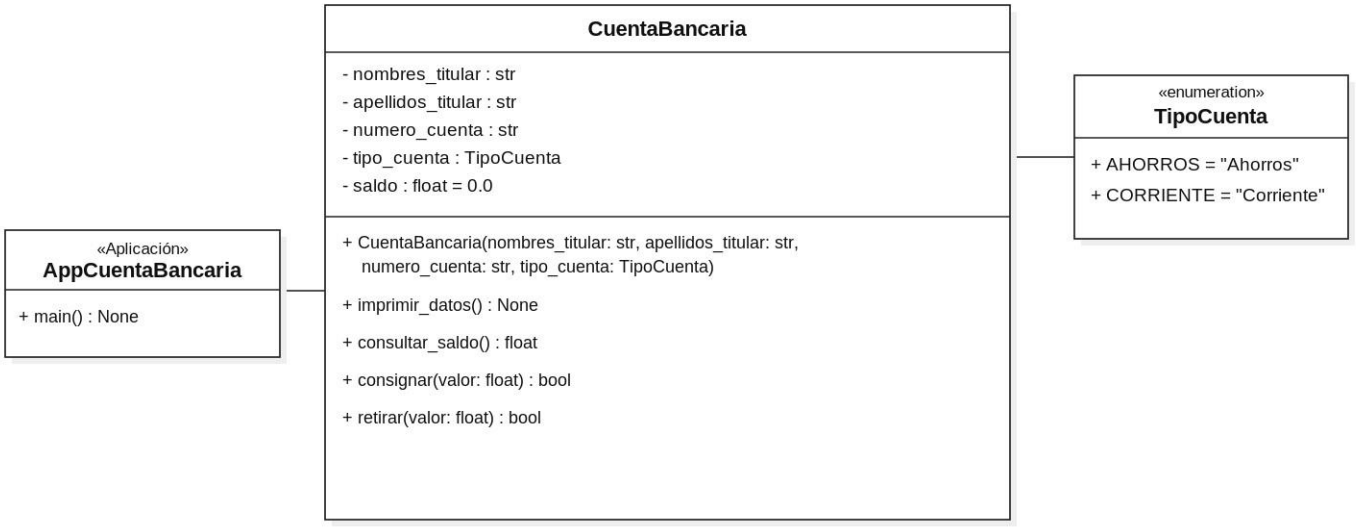
    # 6. Intentar retirar un monto mayor al saldo disponible
```

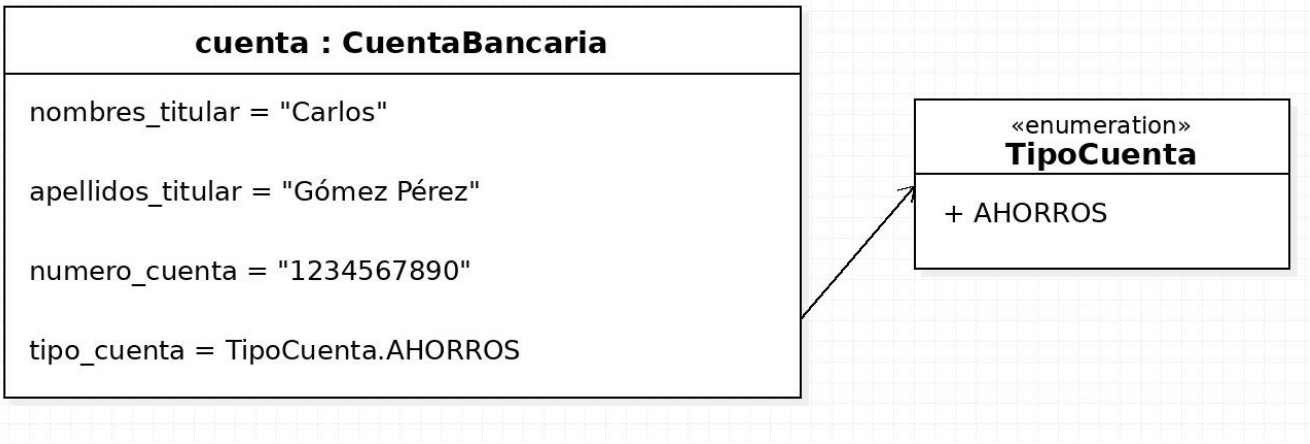
```
    print("\nIntentando retirar $400,000 (monto excedido)...")
cuenta.retirar(400000)

    # 7. Mostrar el estado final de la cuenta     cuenta.imprimir_datos()
```

```
if __name__ == "__main__":
    main()
```

Diagrama de Clase:




cuenta : CuentaBancaria
nombres_titular = "Carlos"
apellidos_titular = "Gómez Pérez"
numero_cuenta = "1234567890"
tipo_cuenta = TipoCuenta.AHORROS
«enumeration»
TipoCuenta
+ AHORROS

URL Repositorio

https://github.com/bcarov-blip/POO_ACTIVIDAD02